

Testovanie mutácií s podporou umelej inteligencie: Intelligentné procesy pre automatizované zabezpečenie kvality

Bc. Ondrej Babinský

Problém: Code coverage neznamená kvalitné testy

- Pri vývoji je ťažké určiť, či testy naozaj overujú správne správanie programu.
- Bežná metrika je **code coverage** - koľko riadkov/funkcií testy vykonajú.
- Problém: coverage hovorí iba to, že sa kód spustil, nie že bol správne otestovaný.
- Aj 100 % coverage môže stále znamenať slabé testy a neodhalené edge cases.
- Preto potrebujeme silnejší spôsob hodnotenia kvality testov.



%

Coverage

Riešenie: Mutation-based testing

- Mutation testing vytvára malé upravené verzie programu – **mutanty**.
- Mutant predstavuje jednoduchú umelú chybu v kóde.
- Testy sa spustia proti mutantovi.
- Ak test zlyhá na mutantovi, mutant je **killed**.
- Ak test stále prejde, mutant **survived** - testy pravdepodobne nepokrývajú daný prípad dostatočne.
- Výsledkom je **mutation score** - pomer zabitých mutantov voči všetkým relevantným mutantom.

```
# pôvodný kód  
return age >= 18
```

```
# mutant  
return age > 18
```

Ak je mutation testing taký dobrý, prečo ho nepoužívame všade?

- Mutation testing je silná technika, ale v praxi má vysokú cenu.
- Na väčších projektoch vie vytvoriť tisíce až desaťtisíce mutantov.
- Každý mutant znamená ďalší beh testov alebo aspoň časť test suite.
- Výsledok: beh môže trvať minúty až hodiny namiesto sekúnd.
- Výber vhodného frameworku nie je jednoduchý.
- Množstvo menej, napr. skipped alebo irrelevant mutantov.

Možnosti riešenia:

Zlepšenie mutation testing nástrojov

- jednoduchší setup, konfigurácia a reporty,
- menej šumu vo výsledkoch, napr. skipped / nerelevantné mutanty.

Targeted mutation testing

- mutovať iba konkrétne súbory alebo riadky,
- vhodné hlavne po refactoringu alebo pri kontrole nedávno zmeneného kódu.

Využitie LLM

- vie pripraviť konfiguráciu, vybrať súbory/riadky/testy a spustiť príkazy bez toho, aby používateľ poznal detaily frameworku.
- navrhovať vlastné mutácie, vyhodnocovať survived mutanty a generovať nové testy, ktoré ich zabijú.

Porovnanie Python Mutation Testing Frameworkov

	Programming Language	Language version requirements	Last Update	OS	Test Runners	Custom mutators
Cosmic Ray	Python	Python 3.xx	2025	Windows, Linux, MacOS	All relevant	Yes
MutMut	Python	Python >=3.6	2025	Linux, MacOS	pytest	No
MutPy	Python	3.3 - 3.11	2019	Windows, Linux, MacOS	pytest, unittest	Yes
XMutant	Python	Python 3.6	2018	Windows, Linux, MacOS	doctests	No
Mutatest	Python	Python 3.7 or 3.8.	2023	Windows, Linux, MacOS	pytest	Yes
Poodle	Python	Python 3.9 - 3.12	03/02/2024	Windows, Linux, MacOS	All relevant	No
	Selective mutating	Parallel mutating	Report format	Ease of setup	Community support	AST Parser
Cosmic Ray	Yes	Yes	Html, console, JSON database dump, XML	Moderate	Moderate	Parso
MutMut	Yes	Yes	custom GUI or .html	Moderate	Moderate	Parso
MutPy	Yes	No	YAML/HTML	Easy	Moderate	ast
XMutant	Yes	Yes	Text	Easy	None	bytecode
Mutatest	Yes	Yes	Text	Moderate	Moderate	ast
Poodle	Yes	Yes	Text, Html, Json	Moderate	Low	ast

Mutation example	Mutation type	mutmut	mutpy	cosmic ray
a + b → a - b	Arithmetic Operator Replacement (AOR)			
x & y → x	Bitwise Operator Replacement (BOR)			
x == y → x != y	Comparison Operator Replacement (COR)			
and → or, True → False	Logical Operator Replacement (LOR)			
-x → +x	Unary Operator Replacement (UOR)			
break → continue	Control Flow Replacement (CFR)			
5 → 0	Numerical constant Replacement (CRP)			
raise A → raise B	Exception Handling Mutations (EXM)			
removes @decorator	Decorator Removal (DEC)			
x → y	Variable Replacement (VRP)			
"hello" → "world"	String mutator			
Deletes entire if block	Block statement mutator			
return x → return None	Method mutator			

Ako najpraktickejší som vyhodnotil Cosmic-ray

Testovanie cosmic-ray

- Testovanie knižnice validators s frameworkom cosmic ray
- Mutácie odhalili chybnú implementáciu validátorov modulu finance (identifikátore cenných dokumentov)
- Checksum sa **vôbec nepočíta**
- Testy prechádzajú, ale kód je chybný
- <https://github.com/python-validators/validators/issues/440>

Cosmic Ray Report

Summary info

Date time: 25/11/2025 01:01:13

Total jobs: 520

Complete: 520 (100.00%)

Surviving mutants: 180 (34.62%)

`def _isin_checksum(value: str):` 1 usage Jovial Joe Jayarson

`check, val = 0, None`

```
for idx in range(12):
    c = value[idx]
    if c >= "0" and c <= "9" and idx > 1:
        val = ord(c) - ord("0")
    elif c >= "A" and c <= "Z":
        val = 10 + ord(c) - ord("A")
    elif c >= "a" and c <= "z":
        val = 10 + ord(c) - ord("a")
    else:
        return False

if idx & 1:
    val += val
```

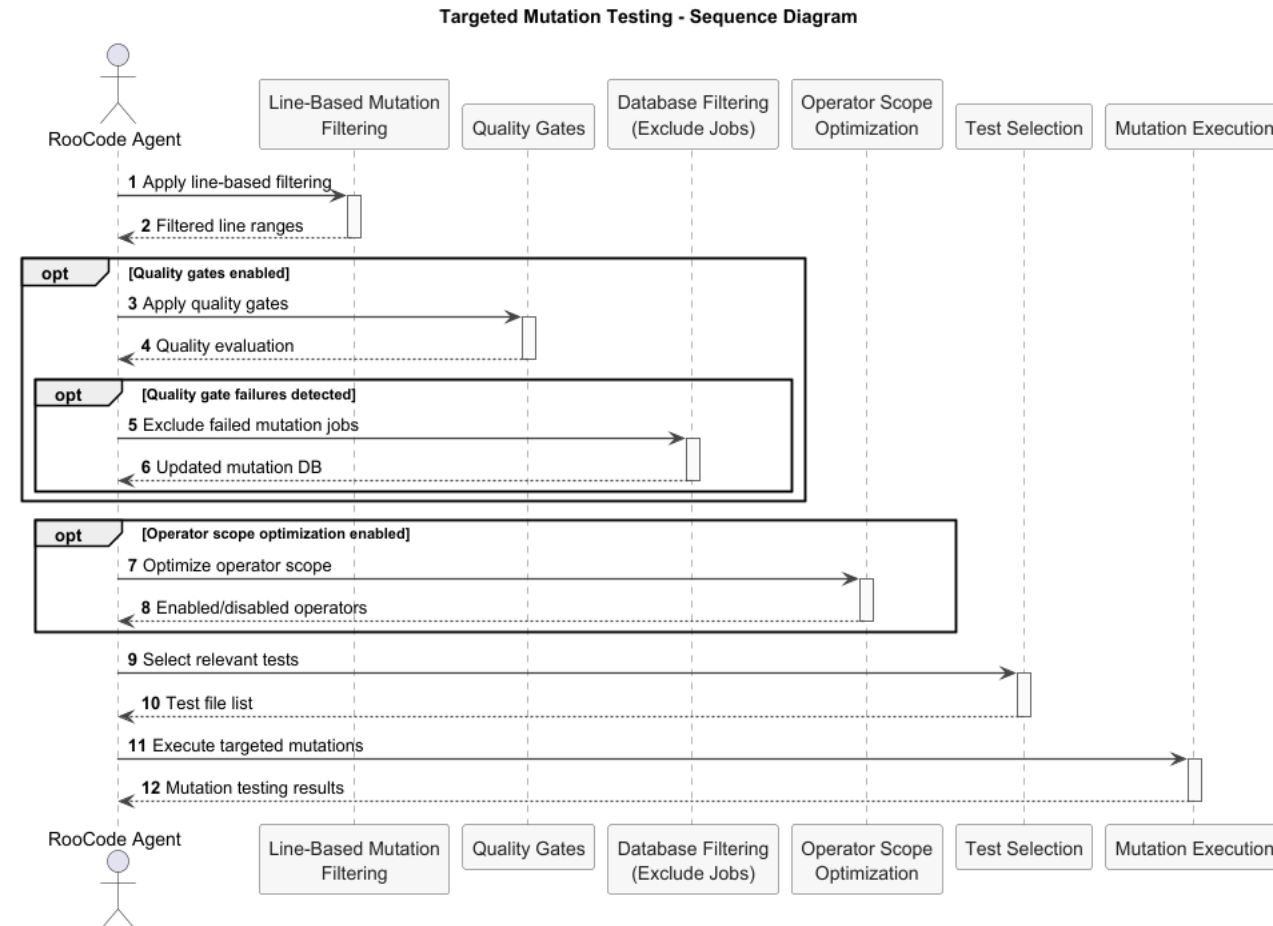
`return (check % 10) == 0`

POC: Targeted mutation testing workflow

- Navrhol som sadu promptov/use-casov, podľa ktorých sa VS code agent riadi.
- Workflow detailne opisuje čo má agent vykonať.
- Cieľ: aby používateľ nemusel poznať detaily frameworku a nespúšťal mutation testing na celom projekte.

Test na python/validators:

- *Vykonal sa len 30/3040 mutácií*
- *Trvanie pár sekúnd*
- *Ináč > 5 min*



Cosmic Ray line-filter

Nová funkcionálna pre cielené mutovanie konkrétnych riadkov

Motivácia:

- pri refaktoringu chceme overiť hlavne zmenené riadky, nie celý projekt.
- Line-filter umožňuje mutovať iba vybrané riadky, čím znižuje počet mutácií, čas behu aj šum vo výsledkoch.

```
[cosmic-ray.filters.line-filter.lines]  
"iban.py" = ["64-67"]  
"card.py" = ["15", "33-38"]
```

Implementácia

- filter načíta konfiguráciu so súbormi a rozsahmi riadkov
- mutácie mimo vybraných rozsahov označí v databáze ako SKIPPED
- Cosmic Ray potom vykoná iba ponechané mutácie
- menej šumu vo výsledkoch, napr. skipped / nerelevantné mutanty.

Skrytie “SKIPPED” mutácií v HTML report

Problém:

HTML report obsahoval množstvo mutácií ktoré boli preskočené (napríklad mojim line-filtrom) a tvorili zbytočný šum.

Riešenie:

- Pridaný nový parameter **--hide-skipped**
- Už v produkcií Cosmic-ray

```
@click.command()  & Ondrej Babinský +2
@click.option( "param_decls: "--only-completed/--not-only-completed", default=False)
@click.option( "param_decls: "--skip-success/--include-success", default=False)
@click.option( "param_decls: "--hide-skipped/--show-skipped", default=False)
@click.argument( "param_decls: "session-file", type=click.Path(dir_okay=False, readable=True, exists=True))
def report_html(only_completed, skip_success, hide_skipped, session_file):
    """Print an HTML formatted report of test results."""
    with use_db(session_file, WorkDB.Mode.open) as db:
        doc = _generate_html_report(db, only_completed, skip_success, hide_skipped)

def _generate_work_item_card(doc, index, work_item, result, skip_success, hide_skipped):
    doc, tag, text = doc.tagtext()
    if hide_skipped and result is not None and result.worker_outcome == "skipped":
        return
```

MCP Server pre mutation workflow

- MCP server = rozhranie medzi Agentom a našimi nástrojmi

Prečo?

- stabilný tool namiesto ručného skladania príkazov
- jednoduché spustenie mutation testingu s line filtrom
- automatická príprava Cosmic Ray configu
- Agent zadá cieľové riadky, tool vykoná workflow



- Agent zavolá MCP tool automaticky
- Vstup: target súbory + riadky + test command
- Tool pripraví config pre line-filter
- Spustí celý Cosmic Ray workflow
- Výstup: total / executed / killed / survived

run line mutation testing workflow on file finance.py lines 25-30

5% · \$0.00

↔ API Request

\$0.0013

 **Roo wants to use a tool on the cosmic-ray-spike MCP server**

 **cosmic-ray-spike**

 **run_line_mutation_workflow**

```
{
  "targets": [
    "repos/validators/src/validators/finance.py:25-30"
  ],
  "config_path": "_spikes/line-mutaton-mcp/config.toml",
  "test_command": "python -m pytest -q repos/validators/tests -k finance"
}
```

 **Roo said**

Executed line mutation testing using `repos/validators/cosmic-ray-config.toml`
(database: `cosmic-ray-sql.sqlite`).

- Total jobs: 573
- Executed mutants: 62
- Killed mutants: 56
- Survived mutants: 6 (9.68%)

Agentic Skills

- Špeciálne inštrukcie pre Agentu
- Načítajú sa pri relevantnom zadaní úlohy

Moje využitie:

- Skill sa loadne pri targeted mutation testingu
- Pomáha Agentovi vybrať minimálny relevantný test command
- Definuje ako sa Agent má správať pri otázkach usera

```
---
name: targeted-mutation-testing
description: Use when user mentions mutation testing,
mutants, Cosmic Ray, line-filter, survived/killed mutants,
or mutation reports.
---

# Mutation testing instructions

If user asks to run mutation testing:
1. identify target file + lines
2. choose minimal test_command
3. call MCP tools:
   - prepare_line_mutation_config
   - execute_line_mutation_testing
4. return total / executed / killed / survived

If user asks for details:
- do not rerun mutation testing
- generate HTML report:
  cr-html --hide-skipped <database_path> > report.html
```

Mutation Testing in Practice: Insights from Open-Source Software Developers

- Autori skúmali, ako sa mutation testing reálne používa v open-source projektoch. Spravili dotazník medzi **104 developermi**

Čo sa tam riešilo:

- prečo developeri používajú mutation testing,
- aké nástroje používajú,
- aké výhody a problémy vnímajú,
- čo im bráni používať mutation testing viac.

Hlavná pointa:

- Developeri mutation testing hodnotia pozitívne, pomáha zlepšovať testy, hľadať slabé miesta a niekedy aj buggy.
- Najväčšie problémy sú performance a settuping

Mutation-Guided LLM-based Test Generation at Meta

- Autori predstavili systém ACH od Meta, ktorý spája mutation testing a LLM
- Cieľom bolo automaticky generovať testy na konkrétne typy chýb, hlavne privacy regresie
- LLM najprv vytvorí mutanta
- následne LLM generuje test, ktorý má túto chybu odhaliť
- systém kontroluje, či test buildne, prejde na pôvodnom kóde a zlyhá na mutantovi
- riešili aj problém ekvivalentných mutantov
- Mutation testing sa dá použiť ako návod pre LLM
- ACH bol nasadený na reálnych Meta platformách a developeri prijali 73 % vygenerovaných testov

Webstránka diplomovej práce:

<https://www.st.fmph.uniba.sk/~babinsky4/mutationtesting/>

***Ďakujem za
pozornosť***